

Event Management System Documentation

System Overview

The Event Management System is a web-based platform that supports 3 main roles:

- 1) **Users**: Browse and book events
- 2) **Organizers**: Create and manage events
- 3) **Admins**: Manage users, organizers, and system operations

➤ The system supports both free and paid events. For paid events, users are redirected to a payment process before confirming their booking.

Definitions

- **User**: A person who uses the system to browse and book events
- **Organizer**: A user with permissions to create and manage events

- **Admin:** A system administrator with full control over users and events
 - **Event:** An activity or occasion that users can attend
 - **Booking:** A reservation made by a user for an event
 - **Payment:** A transaction required for paid events
-

Purpose

- ➔ Define the functional and non-functional requirements of the Event Management System.
 - ➔ This system allows users to browse and book events (free or paid)
 - ➔ Enables organizers to create and manage events, and provides administrators with full control over the platform.
-

Scope

The system includes the following features:

- User registration and authentication
- Event browsing and detailed viewing
- Booking system:
 - Free events → instant booking

- Paid events → redirect to payment gateway
- Event management by organizers
- Admin dashboard for full system control

Problem Statement

Traditional event management often relies on fragmented tools, manual tracking, and disconnected payment systems. This leads to administrative inefficiencies, poor attendee experiences, and difficulties in tracking event analytics. There is a pressing need for a unified platform that handles role-based access, automated notifications, seamless registration, and secure payment processing in one place.

Aim of the Project

- The primary aim of this project is to develop a highly flexible, web-based Event
- Management System that empowers organizers to create, manage, and monitor events
- while providing a frictionless registration and payment experience for attendees.



System Roles and Responsibilities



User

- Register and log in
- Browse available events
- View event details
- Book events:
 - Free → direct booking
 - Paid → complete payment first
- View personal bookings

❖ **Organizer**

- Create and manage events
- Edit or delete events
- Define event details:
 - Title
 - Description
 - Price (free or paid)
 - Date and location
- Monitor bookings for their events

❖ **Admin**

- Manage users (add, delete, block)
- Promote users to organizers

- Manage all events
 - Monitor system activity
 - Manage payments
-

→ Functional Requirements←

1-Authentication

The system shall allow users to

- register
- log in
- log out

2-Event Management

Organizers shall be able to

- create events
- edit events
- delete events

Users shall be able to view all available events

3-Booking System

Users shall be able to

- book free events instantly
- payment for paid events
- confirm booking after successful payment

4-Payment

The system shall

- integrate with a payment gateway
- handle payment success and failure
- link payment status to booking status

5-Admin Panel

Admins shall view all users

- manage user roles
- promote users to organizers
- delete or suspend accounts

→ Non-Functional Requirements ←

1- Security

- Secure passwords using hashing algorithms (BCrypt, Argon2).
- Implement authentication and role-based access control with **Spring Security**.
- Enforce secure **HTTPS/TLS** connections for payment processing.

2- Usability

- Provide an intuitive, user-friendly interface.
- Ensure responsive design that works across modern web browsers.

3- Performance

- Deliver fast API response times.
- Optimize database queries using **Spring Data JPA / Hibernate** (lazy loading, pagination, query tuning).

4- Scalability

- Support horizontal scaling of both API and frontend nodes.
- Use containerization (Docker/Kubernetes) with load balancing.
- Design services to be stateless for easier scaling.

5- Availability

- Ensure high uptime with reliable **SQL Server** hosting.
- Implement robust error handling in backend and frontend.

- Design system architecture for resilience and fault tolerance.

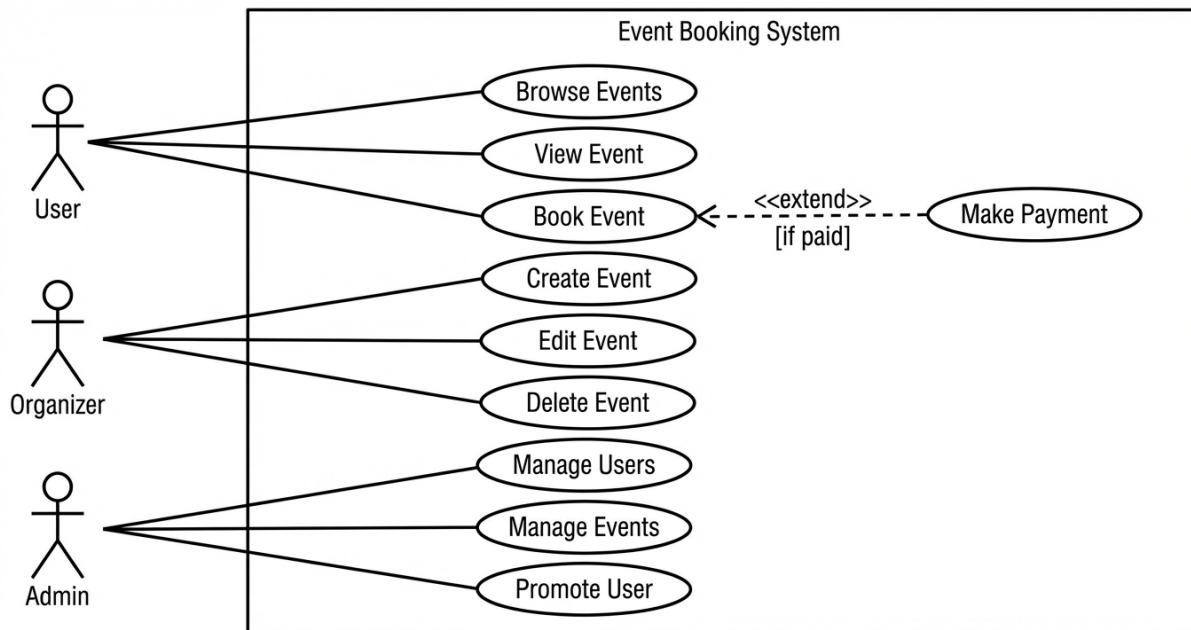
6- Maintainability

- Follow clean architecture principles (separation of controllers, services, repositories).
- Build reusable React components for consistency and testability.
- Apply RESTful API principles with clear resource naming, proper HTTP methods, and versioning

Technologies Used

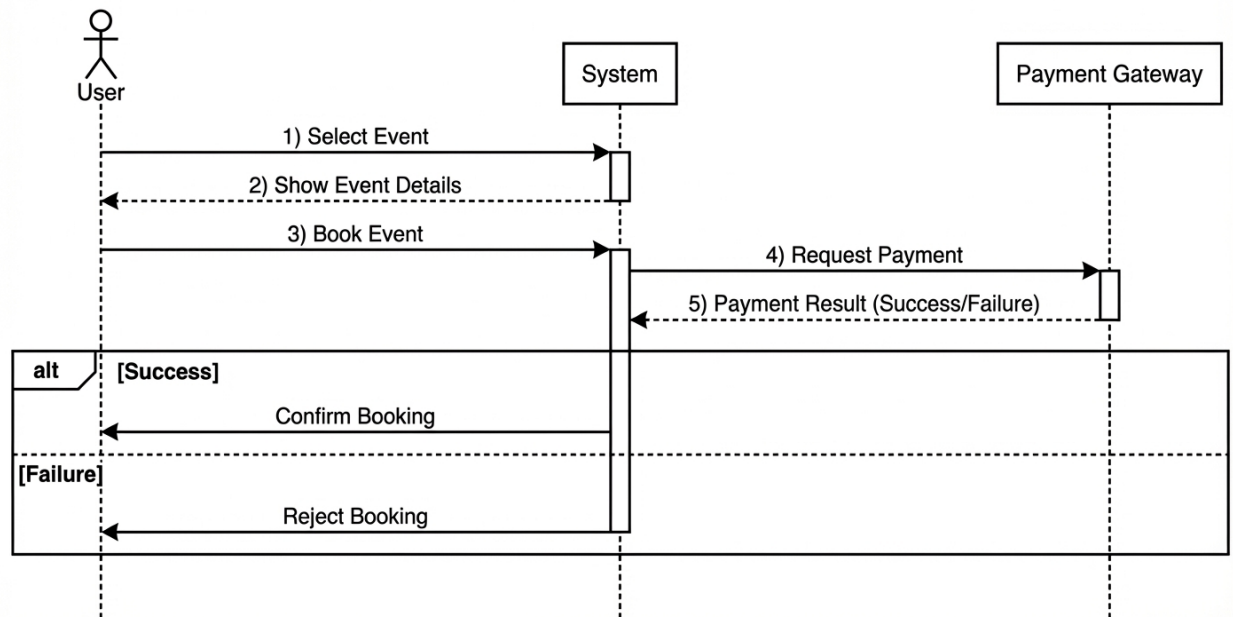
Technology	Purpose / Role
React	Building the front-end user interface for event browsing, registration, dashboards, and interaction.
Java Spring Boot	Handling backend services, RESTful APIs, event creation, business logic, and user management.
Spring Data JPA / Hibernate	Managing database interaction, entity relationships, and CRUD operations using ORM.
Spring Security	Managing authentication, authorization, and role-based access control securely.
Paymob API	Handling payment gateway integration for secure online transactions for paid events.
MySQL	Relational data storage for events, users, registrations, notifications, and transactions.

Use Case Diagram

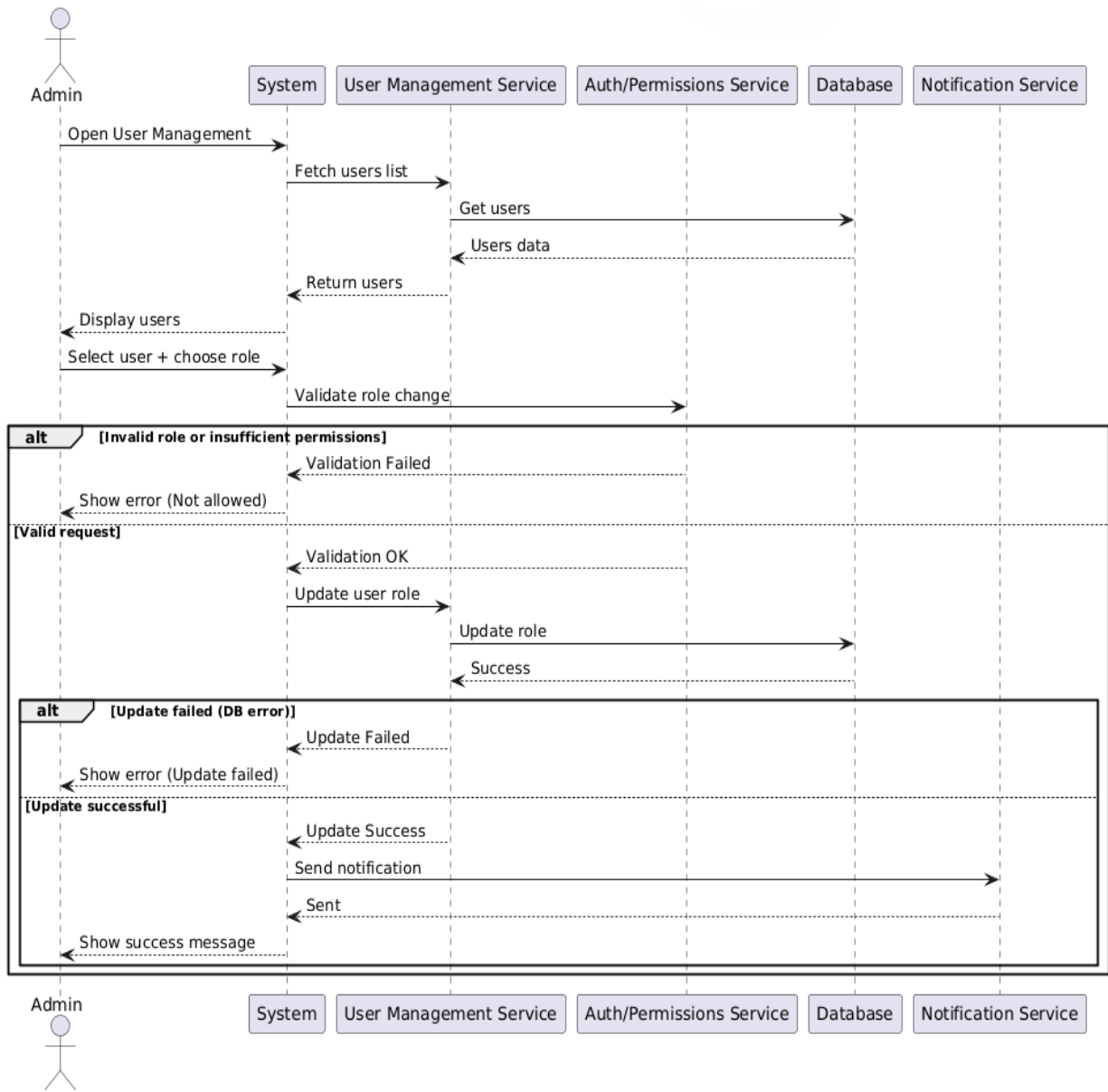


Sequence Diagrams

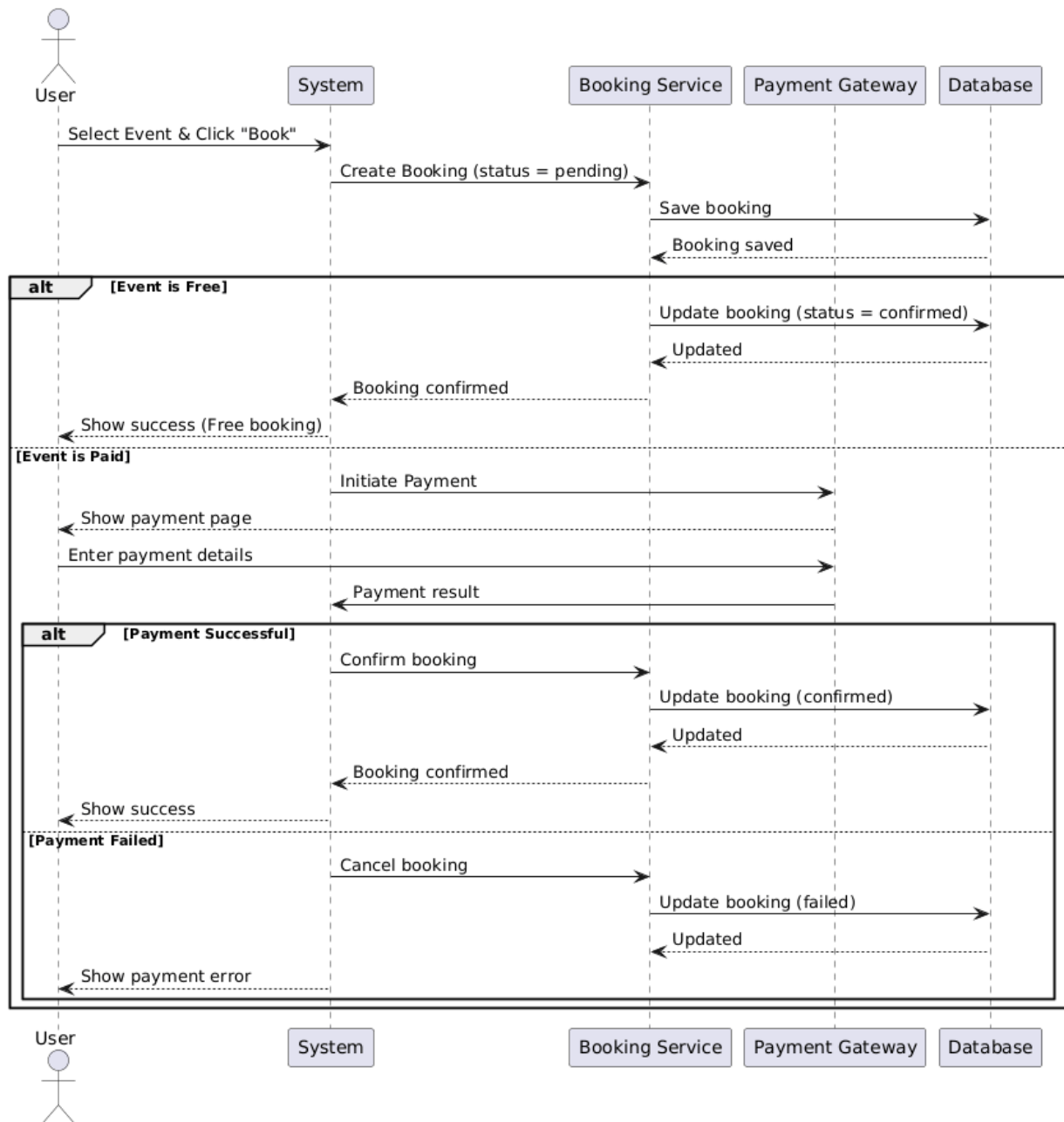
Sequence Diagram: User books a paid event



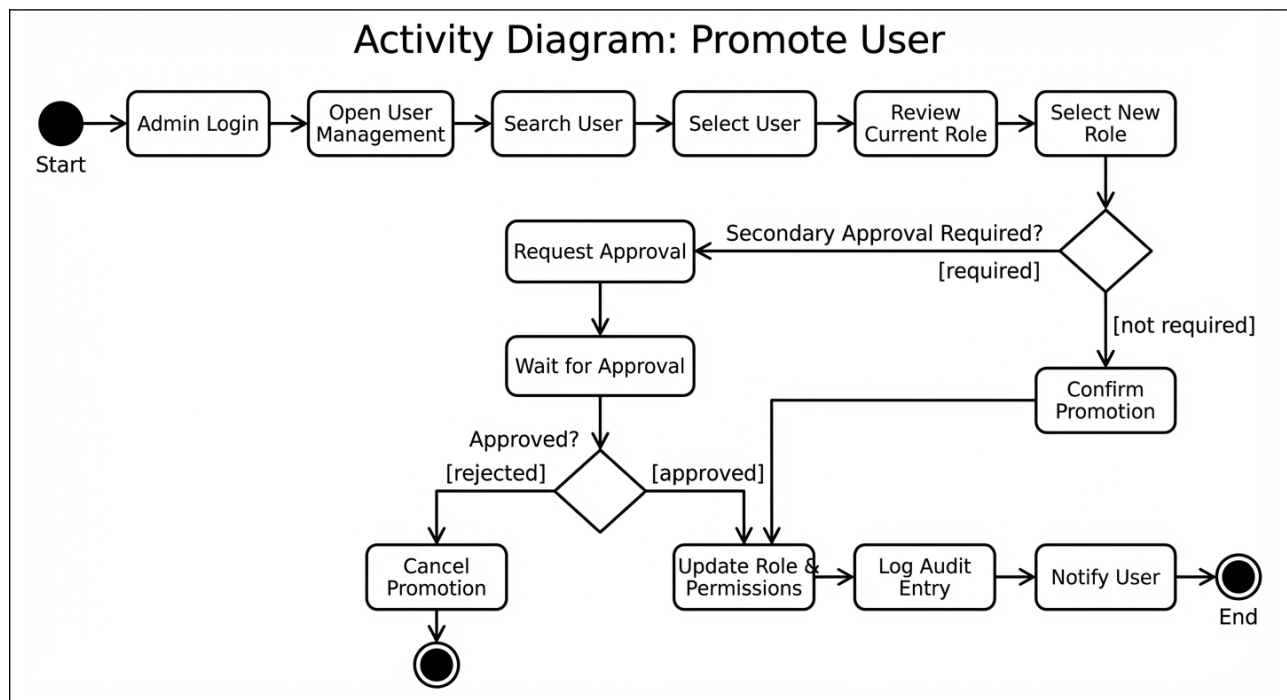
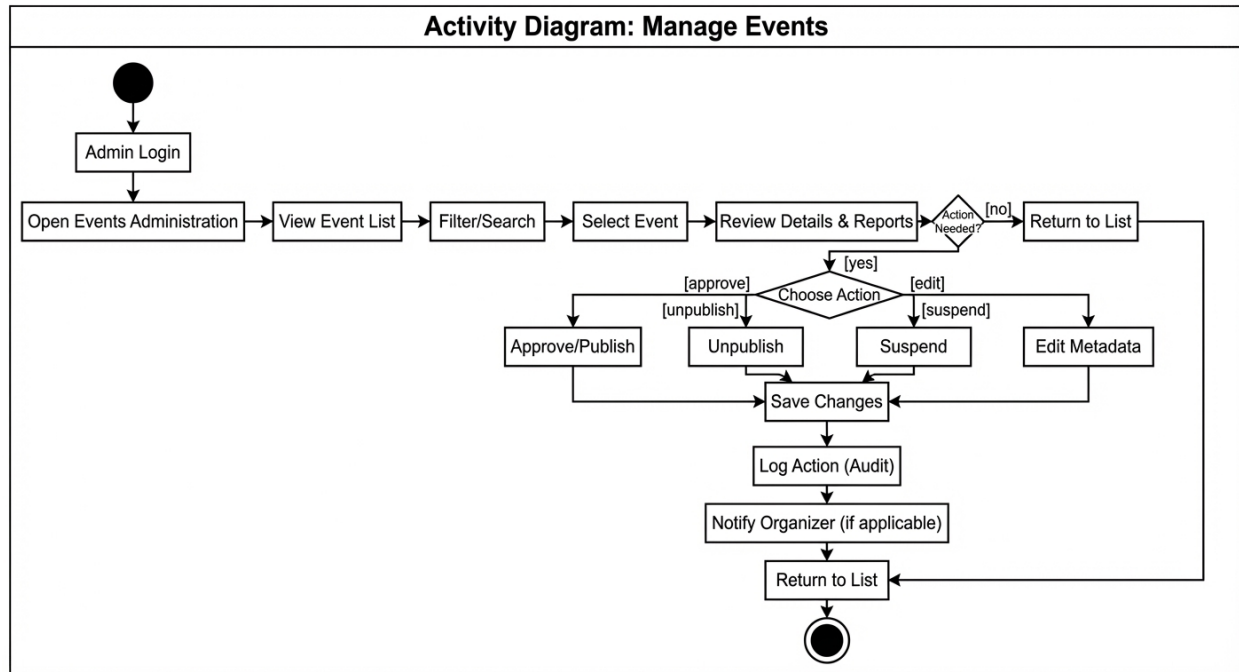
Sequence Diagram: Admin Promotes User



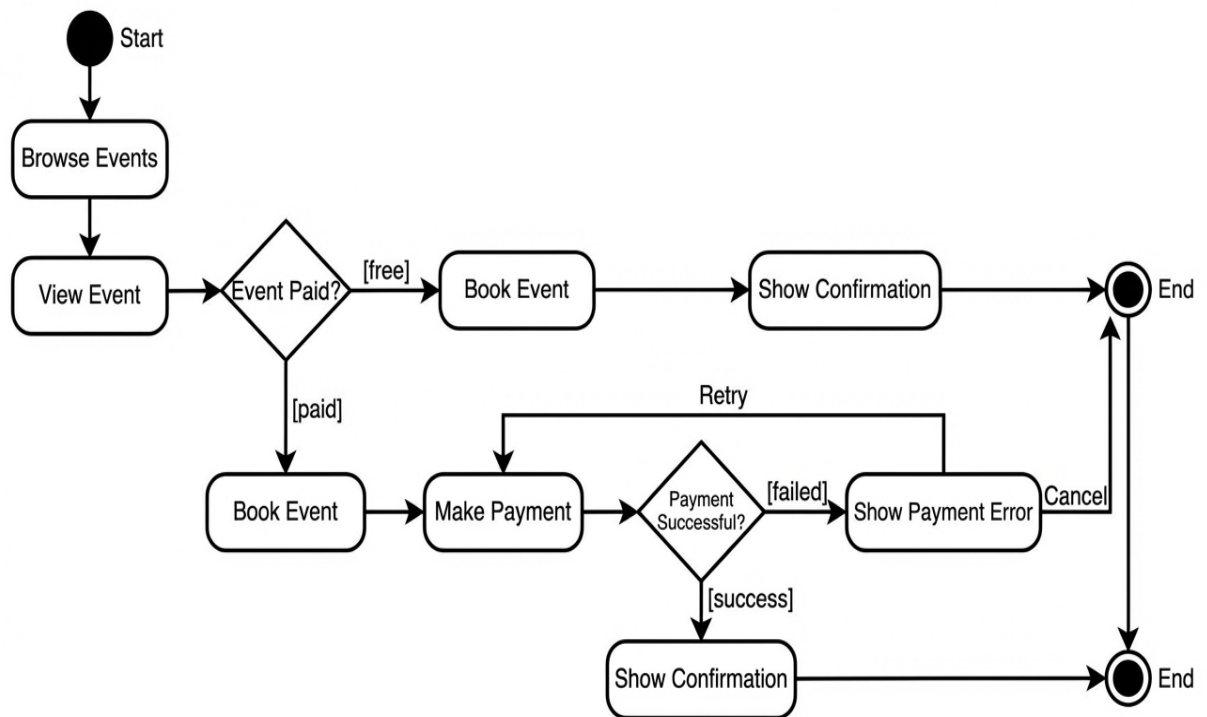
Sequence Diagram: Make Payment



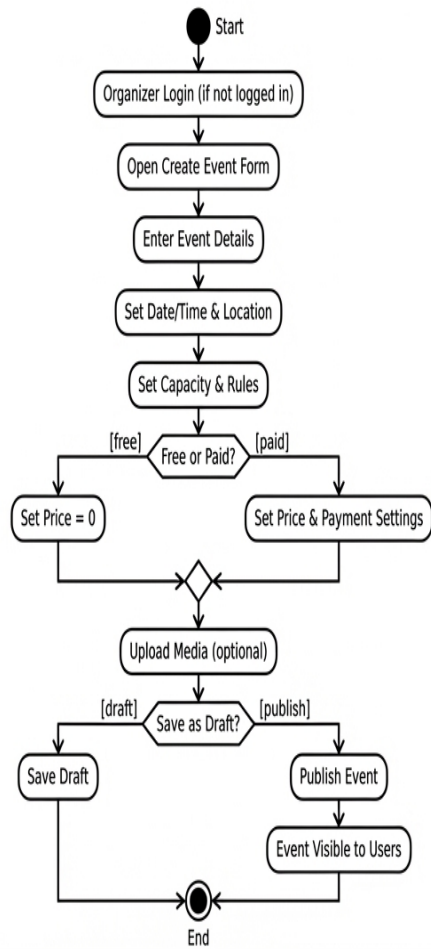
Activity Diagrams



Activity Diagram: Book Event

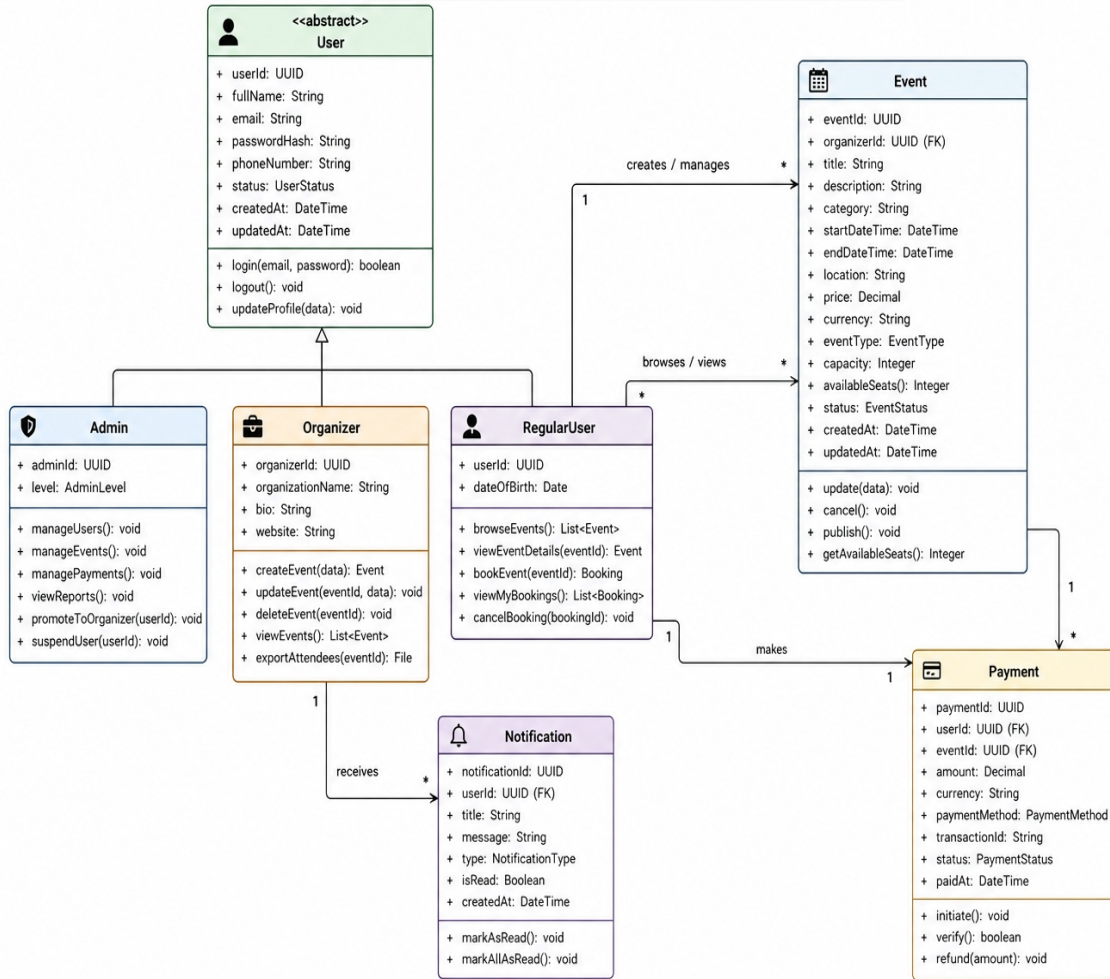


Activity Diagram: Create Event



Class Diagram

Event Management System - Class Diagram



Event Management System – Object Constraint Language (OCL)

Association Consistency Rules

context Organizer inv: self.fullName.size() >= 1 ef 1. organizer = self) — Ensures relation between Organizer and self.phoneNumber.matches("d{10,15}")

context Payment inv: self.logged == (self.passwords->exists(p | p.startsWith('1234567890'); organizedBy — Ensures relation between Payment and Event

Validation Rules

User

context User inv:

- self.fullName.size() >= 1
- self.email.matches('[*!@]@[*!@]_1_[*!@]*')
- self.phoneNumber.matches('d{10,15}')
- self.logged => (self.passwords->exists(p->exists(p | self.passwordHash -p.hash) and self.eneavind)

Organizer

context Organizer inv:

- self.organizationName.size() >= 1
- self.log8uridentical => User.allInstances() -> \User (i) | u.notification1->exists(n in. organizer = self) isEmpty()

RegularUser inv:

context RegularUser inv: self.bookings->forall(b | b.user = self) — User can only book own events

Event

context Event inv:

- self.startDate > now()
- self.endDate > self.startDate
- self.capacity >= 1

— Events must have valid date ranges, capacity, capacity, and price

Payment

context Payment inv:

- self.amount > 0
- self.paymentDate <= now()

— Payment must be valid

Organizer

context RegularUser inv:

- self.bookings->forall() | l. user = self) — User can only book own events

context Event::cancel() =>

- pre: self.startDate > now()
- post: result => null.and-result
- post: result.organizer = self — Petz-status. < Result <= Event'>., false'

Event

context Event inv: self.startDate > now()

- self.endDate > self.startDate
- self.capacity >= 1

— Events must have valid date ranges, capacity, and price

Event

context Event inv:

- self.startDate > now()
- post: self.published = true

Organizer: verify

context Event::refed() >=>

- pre: e.published and e.capacity >= 1
- post: Booking.allInstances() > exists(b | b = size())

Payment operations

context Admin: refund(p : Payment)

- pre: p.status = "Compizted" and p.event.startDate > now() and not p.event.cancelled

Event operations

context Organizer::createEvent(..) : Event

- pre: self.events >forall(e | e -<> self)
- post: result => null and result.published = false and result.organizer = self

Event operations

context Event::publish()

- pre: self.capacity > self bookings > size()
- post: self.published = true

Booking operations

context RegularUser:: bookEvent(e : Event)

- pre: not self.rolidentified
- post: self.identified = true

Payment operations

context Admin::refund(p : Payment) r=>

- pre: p.status = "Compizted" and p.event.startDate > now() and not p.event.cancelled

ERD Schema

